1. Water Jug Problem

Problem Statement

We are given two water jugs:

- One 4-gallon jug
- One 3-gallon jug

Neither jug has any measuring markings. A water pump is available to fill a jug completely. Water can also be poured from one jug into the other or emptied onto the ground.

The objective is to obtain exactly 2 gallons of water in the 4-gallon jug.

Problem Formulation

This can be solved using a state-space search approach.

A state is represented as:

(x, y)

where:

- **x** = amount of water in the 4-gallon jug
- **y** = amount of water in the 3-gallon jug

The initial state is:

(0, 0)

The required goal state is:

(2, y)

where the 4-gallon jug contains exactly 2 gallons.

The possible operations are:

1. Fill the 4-gallon jug completely.
2. Fill the 3-gallon jug completely.

3. Empty the 4-gallon jug.
4. Empty the 3-gallon jug.
5. Pour water from the 4-gallon jug into the 3-gallon jug until either the first jug is empty or the second jug is full.
6. Pour water from the 3-gallon jug into the 4-gallon jug until either the first jug is empty or the second jug is full.

Solution

Starting from the initial state **(0, 0)**:

Step	Operation	4-Gallon Jug	3-Gallon Jug	State
0	Initial state	0	0	**(0, 0)**
1	Fill the 3-gallon jug	0	3	**(0, 3)**
2	Pour 3-gallon jug into 4-gallon jug	3	0	**(3, 0)**
3	Fill the 3-gallon jug again	3	3	**(3, 3)**
4	Pour from 3-gallon jug into 4-gallon jug until 4-gallon jug is full	4	2	**(4, 2)**
5	Empty the 4-gallon jug	0	2	**(0, 2)**
6	Pour the remaining 2 gallons into the 4-gallon jug	2	0	**(2, 0)**

Therefore, the desired state is reached:

Goal State = (2, 0)

Hence, exactly 2 gallons of water are obtained in the 4-gallon jug.

State-Space Representation

The important sequence of states is:

(0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

At the final state, the 4-gallon jug contains exactly **2 gallons** of water.

Method Used

The problem is a state-space search problem. Each possible quantity of water in the two jugs represents a state, and each allowed operation produces a transition from one state to another.

The solution systematically explores valid states until the goal state **(2,0)** is reached.

Result

The required quantity of **2 gallons** is successfully measured in the 4-gallon jug without using any measuring scale or additional device. The solution satisfies all the given assumptions and uses only filling, emptying, and transferring water between the two jugs.

2. Mars Rover as an Intelligent Agent

Problem Statement

A Mars Rover is an autonomous robotic system designed to explore the surface of Mars, analyze the surroundings, and collect and examine samples. Since direct human control is difficult because of the large communication delay between Earth and Mars, the rover needs to make several decisions autonomously.

The Mars Rover can therefore be considered an intelligent agent that perceives its environment through sensors and performs actions using its actuators.

i. Percepts of the Mars Rover

A percept is the information received by an agent from its environment through its sensors.

The Mars Rover can perceive:

- **Images and terrain information** using cameras.
- **Distance and obstacles** using navigation and ranging sensors.
- **Temperature** of the surroundings.
- **Atmospheric conditions** such as pressure and composition.
- **Soil and rock properties** using scientific instruments.
- **Chemical and mineral composition** of collected samples.
- **Position and orientation** using navigation and localization systems.
- **Battery and system status** to monitor its own condition.
- **Weather and environmental conditions** such as dust, wind, and radiation.

These percepts help the rover understand its surroundings and decide what action should be performed next.

ii. Characterization of the Operating Environment

The Mars Rover operates in an environment with the following characteristics:

Property	Characteristic of Mars Rover Environment
Observability	**Partially observable**, because the rover cannot observe everything around it at all times.

Determinism	**Partially deterministic**, since some events such as terrain changes and weather conditions cannot be predicted exactly.
Episodic/Sequential	**Sequential**, because current actions affect future exploration and mission results.
Static/Dynamic	**Dynamic**, because environmental conditions such as dust, temperature, and weather can change.
Discrete/Continuous	**Continuous**, because movement, position, temperature, and other physical properties vary continuously.
Number of agents	Primarily a **single-agent** environment, although communication with mission-control systems and other systems may be involved.
Known/Unknown	**Partially unknown**, because the rover encounters previously unexplored terrain and conditions.

Therefore, the Mars environment is partially observable, partially deterministic, sequential, dynamic, continuous, and partially unknown.

iii. Actions the Agent Can Take

The Mars Rover can perform several actions based on the information obtained from its sensors.

Major actions include:

1. **Move forward or backward** across the Martian surface.
2. **Turn or change direction** to navigate the terrain.
3. **Avoid obstacles** such as rocks, slopes, or other dangerous terrain.
4. **Capture images** of the surrounding environment.
5. **Select a suitable location** for scientific investigation.
6. **Collect soil or rock samples** using its robotic instruments.
7. **Analyze samples** using onboard scientific instruments.
8. **Transmit collected data** to Earth.
9. **Recharge or conserve energy** according to its available power.
10. **Change its mission plan** when unexpected environmental conditions are detected.

These actions allow the rover to explore Mars while achieving its scientific objectives.

iv. Performance Evaluation of the Agent

The performance of the Mars Rover can be evaluated using a **performance measure**. The performance measure determines how successfully the rover is achieving its mission objectives.

Important performance measures include:

- Distance successfully explored
- Number and quality of scientific samples collected
- Accuracy of sample analysis
- Quality and amount of useful scientific data obtained
- Successful navigation and obstacle avoidance
- Energy efficiency
- Communication and transmission of useful information
- Time taken to complete mission objectives
- Minimization of damage or failure
- Successful completion of assigned exploration tasks

A high-performing rover should maximize useful scientific discoveries and exploration while minimizing energy consumption, risks, errors, and unnecessary movement.

v. Suitable Agent Architecture

The most suitable architecture for the Mars Rover is a model-based, goal-based agent with utility-based decision-making.

A simple reflex agent would not be sufficient because the rover operates in a partially observable and changing environment. It needs to maintain information about the environment and use previous percepts to make decisions.

A model-based agent can maintain an internal representation of the environment. This allows the rover to remember previously observed terrain, obstacles, locations, and mission information.

A goal-based agent can select actions according to specific objectives, such as reaching a particular location, collecting a sample, or analyzing a rock.

A utility-based component can help the rover choose between different possible actions by considering factors such as scientific value, distance, energy consumption, safety, and time.

Therefore, a combination of model-based, goal-based, and utility-based characteristics is appropriate for a Mars Rover.

Agent Architecture Flow

The working of the Mars Rover can be represented as:

Environment → Sensors → Percepts → Internal State/Model → Goals & Utility Evaluation → Decision → Actuators → Environment

The rover continuously receives information from the environment, updates its internal state, evaluates possible actions, and performs the most suitable action.

Result

The Mars Rover can be modeled as an intelligent agent that senses its environment, maintains information about its surroundings, makes autonomous decisions, and performs actions to achieve scientific exploration goals. A model-based, goal-based, and utility-based architecture is suitable because the Martian environment is partially observable, dynamic, uncertain, and largely unexplored.

3. 8-Queens Problem

Problem Statement

The 8-Queens problem is a classical AI problem in which 8 queens have to be placed on an 8 × 8 chessboard such that no two queens attack each other.

Since a queen can attack another queen horizontally, vertically, or diagonally, the placement must satisfy the following conditions:

- No two queens should be in the same row.
- No two queens should be in the same column.
- No two queens should be on the same diagonal.

The problem can be formulated as a state-space search problem, where each state represents a possible configuration of queens on the chessboard.

Problem Formulation

Initial State

The initial state is an **empty 8 × 8 chessboard** with no queens placed.

State

A state represents a partial or complete placement of queens on the chessboard.

One convenient representation is to place **one queen in each column** and represent a state using an array:

[r_1, r_2, r_3, ..., r_8]

where r_i represents the row in which the queen is placed in column *i*.

For example:

[1, 3, 5, 7, 2, 4, 6, 8]

represents one queen in each of the eight columns.

Initial State Representation

The initial state can be represented as:

[_, _, _, _, _, _, _, _]

where _ indicates that no queen has yet been placed.

Operators

The main operator is:

Place a queen in an unoccupied column in a row where it does not attack any previously placed queen.

After placing a queen, the next column is considered.

Goal State

A goal state is reached when all **8 queens have been placed** and none of them attack another queen.

Thus, the goal condition is:

8 queens placed + no row conflicts + no column conflicts + no diagonal conflicts

Search Strategy

A suitable strategy for solving the 8-Queens problem is **Backtracking Search**, which is a systematic form of depth-first search.

The algorithm places queens one column at a time. If a queen can be placed safely, the search proceeds to the next column. If no safe position is available in a particular column, the algorithm goes back to the previous column and changes the position of the previously placed queen.

This avoids exploring configurations that are already known to violate the constraints.

Steps of the Search

1. Start with an empty chessboard.
2. Place a queen in the first column.
3. Move to the next column.
4. Check whether a row is safe for placing the queen.
5. Check whether the new queen shares a diagonal with any previously placed queen.

6. If the position is safe, place the queen and continue.
7. If no safe position is available, backtrack to the previous column.
8. Change the position of the previous queen and continue searching.
9. Repeat until all 8 queens are placed safely.

One Valid Solution

One valid arrangement of 8 queens is represented by:

[1, 5, 8, 6, 3, 7, 2, 4]

This means:

Column	Row
1	1
2	5
3	8
4	6
5	3
6	7
7	2

8	4

The corresponding chessboard configuration is:

	C1	**C2**	**C3**	**C4**	**C5**	**C6**	**C7**	**C8**
R 1	Q	.	.	.	.	.	.	.
R 2	.	.	.	.	.	.	Q	.
R 3	.	.	.	.	Q	.	.	.
R 4	.	.	.	.	.	.	.	Q
R 5	.	Q	.	.	.	.	.	.
R 6	.	.	.	Q	.	.	.	.
R 7	.	.	.	.	.	Q	.	.

R 8	.	.	Q	.	.	.	.	.

Here, **Q** represents a queen and **.** represents an empty square.

Constraint Checking

For every newly placed queen, the following conditions are checked:

Row condition:

No two queens should have the same row.

Column condition:

Since one queen is placed in each column, there is no column conflict.

Diagonal condition:

For two queens at positions (r_1, c_1) and (r_2, c_2), they are on the same diagonal if:

$$|r_1 - r_2| = |c_1 - c_2|$$

Therefore, a new queen is placed only when this condition is false for every previously placed queen.

Result

The 8-Queens problem can be formulated as a constraint-based state-space search problem. By placing one queen at a time and using backtracking whenever a constraint is violated, a valid configuration containing **8 non-attacking queens** can be obtained.

The valid solution:

[1, 5, 8, 6, 3, 7, 2, 4]

satisfies the required conditions, as no two queens share the same row, column, or diagonal.

4. OLA Cab Booking Problem

Problem Statement

A customer wants to travel from one location to another using the OLA Cab booking mobile application. The customer can select a suitable cab type such as Mini, Micro, Sedan, Shared, Prime, etc., depending on comfort, cost, and travel requirements.

The task is to identify the suitable **problem-solving agent** for this situation and write the corresponding pseudocode.

Problem-Solving Agent

A goal-based agent is suitable for the OLA cab booking problem.

The main goal of the customer is to reach the destination successfully. The agent can consider different cab options and select one that satisfies the customer's requirements.

The agent can evaluate factors such as:

- Pickup location
- Destination
- Available cab types
- Estimated fare
- Estimated travel time
- Distance
- Customer's comfort preference
- Availability of the selected cab

The agent then selects an appropriate cab and generates a route to reach the destination.

Why Goal-Based Agent is Suitable

A goal-based agent is appropriate because it takes actions to achieve a specific goal.

In this problem, the goal is:

Reach the selected destination from the pickup location using a suitable available cab.

The agent does not simply react to one particular input. It considers the available alternatives and selects an action that helps achieve the desired destination.

For example, if the customer wants a comfortable ride, the agent can prefer a Sedan or Prime cab. If the customer wants a lower-cost option, it can select Mini, Micro, or Shared depending on availability.

Problem Formulation

Initial State

The customer is at the pickup location and has entered the desired destination.

Actions

The possible actions include:

1. Enter pickup location.
2. Enter destination.
3. View available cab types.
4. Compare cab options.
5. Select a suitable cab.
6. Book the cab.
7. Travel through the selected route.
8. Reach the destination.

Goal State

The goal state is:

Customer reaches the destination successfully using the selected cab.

Constraints

The agent should consider:

- Cab availability
- Customer's budget
- Customer's comfort preference
- Estimated travel time
- Distance to destination
- Route conditions

Pseudocode

```
BEGIN

Input pickup_location
Input destination

Find available cabs

IF no cabs are available THEN
   Display "No cabs available"
   END
END IF

Display available cab types

FOR each available cab
   Calculate estimated fare
   Calculate estimated travel time
   Check customer comfort preference
END FOR

Select the most suitable cab

Display selected cab and estimated fare

IF customer confirms booking THEN
   Book the selected cab
   Assign driver
   Generate route from pickup location to destination

   WHILE destination is not reached
      Follow the route
      Monitor current location and route
      IF route condition changes THEN
         Select an alternative route
      END IF
   END WHILE

   Display "Destination reached"
ELSE
   Cancel booking
```

END IF

END

Working of the Agent

The working process can be represented as:

Customer Input → Identify Pickup & Destination → Find Available Cabs → Compare Options → Select Suitable Cab → Book Cab → Generate Route → Travel → Reach Destination

The agent continuously uses the available information to make decisions while the customer is travelling.

Result

The **goal-based agent** is suitable for the OLA cab booking problem because it works toward the specific goal of reaching the destination. It considers available cab types, cost, comfort, availability, and travel conditions before selecting an appropriate action.

The pseudocode demonstrates how the agent can accept the customer's requirements, select a suitable cab, book it, generate a route, and complete the journey.

5. Uniform Cost Search for Logistics Delivery Network

Problem Statement

A logistics company has a network of warehouses connected by routes. Each route has an associated travel cost based on factors such as fuel consumption and travel time.

The company needs to determine the **least-cost path from the starting warehouse S to the destination warehouse G** using the **Uniform Cost Search (UCS)** algorithm.

Problem Formulation

Initial State

The initial state is the starting warehouse:

S

Goal State

The goal state is the destination warehouse:

G

Path Cost

Each edge between two warehouses has an associated cost.

The total cost of a path is the sum of the costs of all routes travelled along that path.

For example:

S → A → G

has total cost:

Cost(S,A) + Cost(A,G)

Search Strategy

Uniform Cost Search always expands the node with the **lowest cumulative path cost** from the starting node.

It uses a priority queue in which the node having the smallest path cost is given the highest priority.

Therefore:

Priority = cumulative path cost from S

UCS Algorithm

The steps of Uniform Cost Search are:

1. Start from the initial node **S**.
2. Insert S into the priority queue with path cost 0.
3. Select the node with the smallest cumulative path cost.
4. If the selected node is **G**, the goal has been reached.
5. Otherwise, expand the selected node.
6. Calculate the new cumulative cost for each neighbouring node.
7. Add the neighbouring nodes to the priority queue.
8. Continue selecting the node with the lowest cumulative cost.
9. Repeat until the goal node G is selected.

Pseudocode

```
BEGIN

Create an empty priority queue

Insert starting node S with cost 0

WHILE priority queue is not empty

   Remove the node with the lowest path cost

   IF node is G THEN
      Return the path from S to G
      Return the total path cost
   END IF

   IF node has not been visited THEN

      Mark node as visited
```

```
        FOR each neighbouring node

            Calculate:
            new_cost = current_cost + route_cost

            Insert neighbouring node into
            priority queue with new_cost

        END FOR

    END IF

END WHILE

Return "No path exists"

END
```

Working of Uniform Cost Search

Suppose there are multiple possible routes from **S** to **G**.

UCS does not simply select the route having the fewest number of edges. Instead, it compares the total accumulated cost of each available path.

For example:

- Path 1: S → A → G
- Path 2: S → B → G
- Path 3: S → C → D → G

The algorithm calculates the total cost of each path and always expands the currently available path with the smallest cumulative cost.

Therefore, a path containing more edges can still be selected if its total cost is lower.

Important Property

Uniform Cost Search is **optimal when all step costs are non-negative**.

This makes UCS suitable for the logistics problem because the route costs represent quantities such as fuel and travel time, which are non-negative.

When the goal node **G** is removed from the priority queue as the minimum-cost node, the path found to G is the least-cost path.

Result

Uniform Cost Search can be used to determine the minimum-cost route from warehouse S to warehouse G. It systematically expands the warehouse having the lowest cumulative travel cost and continues until the destination is reached.

The final output consists of:

Least-cost path from S to G

and

Total minimum path cost

The actual numerical path and cost depend on the weighted graph provided in the question.